

密码学实验代码速记手册

Makefile

- 由一系列规则构成
- 每条规则包含目标、依赖、指令

伪随机数概念：与随机数不可区分（伪随机性），可重现

安全性评价标准：

全周期：任意数都可能被生成

不可预测：无法基于已有序列推断出下一个随机数

伪随机数生成器 —— 线性同余

$$X_{i+1} = aX_i + c \mod m$$

参数：乘数a，增量c，模数m

种子：

$$X_0$$

安全性：无法满足全周期，不具备可证明安全性

伪随机数生成器 —— BBS

参数选择：

- 选择素数

$$p, q$$

，满足

$$p \mod 4 = q \mod 4 = 3$$

，

$$N = p \times q$$

- 选择种子

$$s$$

，满足

和

N

互素

迭代运算：

$$X_0 = s, X_i = X_{i-1}^2 \mod N$$

，选择重要比特位（如最低位，奇校验位，偶校验位等）

安全性：基于**大数难分解**困难问题

数字签名特征：完整性，不可伪造性，不可抵赖性

- RSA_PKCS1_PADDING：消息长度小于RSA_size(rsa)-11
- RSA_PKCS1_OAEP_PADDING：消息长度小于RSA_size(rsa)-41
- RSA_NO_PADDING：消息长度等于RSA_size(rsa)

RSA 解密函数 padding 方式与加密函数一致

对称密码

加解密使用**相同**密钥

优点：加解密速度快，密钥管理简单，适合一对一通信

缺点：密钥分发困难，不适合一对多加密传输

流密码

对称加密，基于密钥产生相同伪随机流，与明文按**位**异或加密

优点：低错误传播，硬件实现简单

缺点：扩散度低

RC4

具有**可变密钥长度**（1 - 255 字节）

KSA 算法：基于 K 置换状态数组

PRNG 算法：扩充状态数组，加密明文数据

分组密码

对称加密，将明文**分组**，并将每个分组作为整体进行加解密

Feistel 密码结构：构造分组密码的密码结构

- 扩散：明文每一位影响密文许多位
- 混淆：隐藏密文与密钥统计关系

DES

分组长度 64 位

有效密钥长度 56 位

基于 Feistel 网络结构

明文→初始置换→16轮 Feistel 网络→逆置换→密文

分组密码应用模式

| ECB

处理包含**多个分组长度**的数据的加解密操作

特点：并行访问，并行解密，随机访问，安全隐患

| CBC

依赖初始向量IV（公开，随机）

特点：串行加密，并行解密，随机访问

| CTR

依赖Nonce（公开，随机，非重复）

特点：并行加密，并行解密，随机访问，预加密

哈希函数

将任意长度数据内容映射为固定长度哈希值

特点：单向性，抗碰撞

应用：数据完整性检测

	SHA1	SHA224	SHA256	SHA384	SHA512
消息摘要长度	160	224	256	384	512
消息长度	$< 2^{64}$	$< 2^{64}$	$< 2^{64}$	$< 2^{128}$	$< 2^{128}$
分组长度	512	512	512	512	512
字长度	32	32	32	64	64
步骤数	80	64	64	80	80

SHA1

预处理：填充，分块512

分割字：将每个分块分割并扩展，共计产生 80 个字（字长 32 比特位）

定义5个寄存器，基于字内容进行变换，最终形成SHA1哈希

1. 数论基础算法

1.1 二进制与十进制转换

记忆口诀：

- 转十进制：左移乘2，或运算加1。
- 转二进制：取模得低位，右移除2，最后要翻转。

C++

```
#include <iostream>
#include <string>
#include <algorithm>
using namespace std;

// 【二转十】 输入二进制字符串，返回整数
unsigned int btod(string input) {
    unsigned int res = 0; // 初始化结果为0
    for (char c : input) { // 遍历字符串的每一位
        res <<= 1; // 结果左移1位（等同于乘以2）
        if (c == '1') res |= 1; // 如果当前字符是'1'，将结果最低位置为1（等同于加1）
    }
}
```

```

    }
    return res;
}

// 【十转二】 输入整数，返回二进制字符串
string dtob(unsigned int input) {
    if (input == 0) return "0"; // 特判0的情况
    string res = ""; // 初始化结果字符串
    while (input > 0) { // 只要数字还没除完
        if (input % 2 == 0) res += "0"; // 偶数，最低位是0
        else res += "1"; // 奇数，最低位是1
        input >>= 1; // input右移1位（等同于除以2）
    }
    reverse(res.begin(), res.end()); // 算出来是反的，需要翻转一下
    return res;
}

```

1.2 模指数运算 (快速幂)

记忆口诀： `res` 乘底数， `base` 自己滚，指数右移。

实现（分治）：

```

long long mod_pow(long long a, long long b, long long p)
{
    //返回条件
    if (b == 0) return 1;
    long long r = mod_pow(a, b/2, p);
    if (b % 2) {
        r = r * r * a % p;
    }
    else {
        r = r * r % p;
    }
    return r;
}

```

C++

```

// 计算 (a^e) % p
unsigned int mod_exp(unsigned int a, unsigned int e, unsigned int p) {
    unsigned int res = 1; // 结果初始化为1
    a %= p; // 防止底数一开始就比p大
    while (e > 0) { // 当指数大于0时循环

```

```

        if (e & 1)                // 如果指数当前最低位是1 (e是奇数)
            res = (res * a) % p;    // 把当前的底数乘到结果里，并取模
        a = (a * a) % p;          // 底数自己平方（翻倍），并取模
        e >>= 1;                  // 指数右移1位（除以2）
    }
    return res;
}

```

1.3 埃氏筛 (素数筛选)

记忆口诀：由小到大，遇素数，杀倍数。

C++

```

#include <vector>
// 判断 a 是否为素数（生成法）
bool prime_test(unsigned int a) {
    if (a < 2) return false;      // 小于2都不是素数

    // 创建标记数组，a+1个位置，初始化为0（0代表素数，1代表合数）
    vector<int> is_composite(a + 1, 0);

    // 从2开始遍历，只需要遍历到 sqrt(a)
    for (int i = 2; i * i <= a; i++) {
        if (!is_composite[i]) {    // 如果 i 标记为0（是素数）
            // 从 i*i 开始，把 i 的所有倍数都标记为合数
            for (int j = i * i; j <= a; j += i) {
                is_composite[j] = 1; // 标记为1（合数）
            }
        }
    }
    return !is_composite[a];      // 如果是0返回true，是1返回false
}

```

1.4 Miller-Rabin 素性检测

记忆口诀：拆分 $n - 1$ ，随机选 a ，算 $v = a^q$ ，二次探测 v^2 。

C++

```

// 依赖上面的 mod_exp 函数，这里简写为 qp
// 检测 n 是否为素数，重复 repeat 次
bool miller_rabin(int n, int repeat) {
    // 排除小于2和偶数(除了2)
    if (n < 2 || (n % 2 == 0 && n != 2)) return false;
}

```



```

if (n == 2 || n == 3) return true;

// 1. 分解  $n-1 = q * 2^k$ 
int q = n - 1, k = 0;
while ((q & 1) == 0) {           // 只要 q 是偶数
    q >>= 1;                     // q 除以 2
    k++;                         // k 加 1
}

// 2. 进行多次测试
while (repeat--) {
    int a = rand() % (n - 3) + 2; // 随机底数 a, 范围 [2, n-2]
    int v = qp(a, q, n);         // 计算  $v = a^q \% n$ 

    if (v == 1 || v == n - 1) continue; // 如果 v 是 1 或 n-1, 本轮通过

    // 3. 二次探测序列
    int j = 1;
    for (; j < k; j++) {         // 最多做 k-1 次平方
        v = (long long)v * v % n; // v 变成  $v^2$ 
        if (v == n - 1) break;   // 遇到 n-1, 通过
    }
    if (j == k) return false;    // 循环完了都没遇到 n-1, 是合数
}
return true;                    // 通过所有测试, 大概率是素数
}

```

1.5 扩展欧几里得 & 逆元

记忆口诀：递归互换 a, b ，回溯计算 $y -= (a/b)*x$ 。

C++

```

// 求解  $ax + by = \gcd(a, b)$ 
void exgcd(int a, int b, int& x, int& y) {
    if (b == 0) {                 // 递归终止条件
        x = 1; y = 0;             // b为0时, x=1, y=0
    } else {
        exgcd(b, a % b, y, x);    // 递归调用, 注意 x 和 y 参数互换
        y -= (a / b) * x;         // 回溯公式:  $y = y' - \text{floor}(a/b)*x'$ 
    }
}

// 求 a 在模 m 下的逆元
int reverse_mod(int a, int m) {

```

```

int x, y;
exgcd(a, m, x, y);           // 调用扩展欧几里得
return (x % m + m) % m;      // 防止算出负数，转为正余数
}

```

2. 伪随机数生成器

2.1 LCG (线性同余)

公式： $S = (aS + c) \bmod m$

C++

```

unsigned int s;                // 全局种子变量
void lbsrand(unsigned int seed) { s = seed; } // 设置种子

unsigned int lcg_rand() {
    unsigned int a = 1103515245, c = 12345;
    unsigned int m = 2147483648;    // 模数 2^31
    // 简单公式，实际考试建议加 (unsigned long long) 防止溢出
    s = (a * s + c) % m;            // 更新 s
    return s;                      // 返回新随机数
}

```

2.2 BBS 生成器

公式： $s = s^2 \bmod n$

提取： 最低位 (&1) 或 奇偶性 (count()%2)。

C++

```

unsigned int bbs_rand(int flag = 0) {

    unsigned int p = 11, q = 19;
    unsigned int N = p * q;
    unsigned int res = 0, bit = 0;
    unsigned int x = s; // 使用当前状态
    s = s * s % N; // 更新状态
    for (int i = 0; i < 32; i++) {

        x = x * x % N; // BBS核心：平方取模
    }
}

```



```

    if (flag == 0) {
        bit = x & 1;  // 最低位
    }
    //将整数 x 转换为32位的二进制表示
    // .count返回该位集合中值为1的位的个数（即汉明重量）
    // 计算上一步结果的奇偶性
    else if (flag == 1) {
        bit = bitset<32>(x).count() % 2;  // 奇校验
    }
    else if (flag == 2) {
        bit = (bitset<32>(x).count() % 2) ^ 1;  // 偶校验
    }
    res = (res << 1) | bit;
}

return res;
}

```

3. OpenSSL 大数运算

头文件：#include <openssl/bn.h>

流程：New -> Dec2Bn -> Calc -> Bn2Dec -> Free。

C++

```

// 示例：计算大数模指数 (a^e mod m)
string bn_mod_exp(string a, string e, string m) {
    // 1. 创建大数对象和上下文
    BIGNUM *b_a = BN_new();
    BIGNUM *b_e = BN_new();
    BIGNUM *b_m = BN_new();
    BIGNUM *res = BN_new();
    BN_CTX *ctx = BN_CTX_new();

    // 2. 字符串转 BIGNUM
    BN_dec2bn(&b_a, a.c_str());
    BN_dec2bn(&b_e, e.c_str());
    BN_dec2bn(&b_m, m.c_str());

    // 3. 核心计算：res = a^e % m
    BN_mod_exp(res, b_a, b_e, b_m, ctx);
    // 若求逆元用：BN_mod_inverse(res, b_a, b_m, ctx);
}

```

```

// 4. BIGNUM 转回 字符串
char* str = BN_bn2dec(res);      // 转成 char*
string result(str);              // 转成 C++ string

// 5. 释放内存
OPENSSL_free(str);               // 释放字符串
BN_free(b_a); BN_free(b_e); BN_free(b_m); BN_free(res);
BN_CTX_free(ctx);               // 释放上下文

return result;
}

```

4. RSA 加密与签名

头文件：rsa.h, pem.h, err.h

原则：

- **加密/验签** -> 用**公钥** (**PUBKEY**)
- **解密/签名** -> 用**私钥** (**Private**)

```

bool load_RSA_keys()
{
    FILE* private_key_file = fopen(PRIVATE_KEY_PATH, "r");

    rsa_private_key = PEM_read_RSAPrivateKey(private_key_file, NULL, NULL,
    NULL);
    fclose(private_key_file);

    ....同上
}

string RSA_Encryption(string plaintext)
{
    load_RSA_keys();
    int rsa_len = RSA_size(rsa_public_key);
    unsigned char* encrypted = (unsigned char*)malloc(rsa_len);

    int result = RSA_public_encrypt(
        plaintext.length(),
        (unsigned char*)plaintext.c_str(),

```

```

        encrypted,
        rsa_public_key,
        RSA_PKCS1_PADDING
    );

    if (result == -1)
    {
        fprintf(stderr, "RSA encrytion failed\n");
        free(encrypted);
        return "";
    }

    string ciphertext = base64_encode(encrypted, result);
    // string ciphertext(reinterpret_cast<char *>(encrypted), result);
    free(encrypted);
    return ciphertext;
}

string RSA_Decryption(string ciphertext)
{
    load_RSA_keys();

    size_t decoded_length;
    unsigned char* decrypted_data = base64_decode(ciphertext,
&decoded_length);

    int rsa_len = RSA_size(rsa_private_key);
    unsigned char* decrypted = (unsigned char*)malloc(rsa_len);

    int result = RSA_private_decrypt(
        decoded_length,
        decrypted_data,
        decrypted,
        rsa_private_key,
        RSA_PKCS1_PADDING
    );

    if (result == -1)
    {
        fprintf(stderr, "RSA decryption failed\n");
        free(decrypted);
        free(decrypted_data);
        return "";
    }
}

```

```

    string plaintext(reinterpret_cast<char*>(decrypted), result);
    free(decrypted); free(decrypted_data);
    return plaintext;
}

string RSA_signature_signing(string input)
{
    load_RSA_keys();
    unsigned char hash[SHA256_DIGEST_LENGTH];
    SHA256((unsigned char*)input.c_str(), input.length(), hash);

    unsigned char* signature = (unsigned
char*)malloc(RSA_size(rsa_private_key));

    unsigned int signature_len;
    if (RSA_sign(NID_sha256, hash, SHA256_DIGEST_LENGTH, signature,
&signature_len, rsa_private_key) != 1)
    {
        fprintf(stderr, "RSA signing failed\n");
        free(signature);
        return "";
    }

    string signed_message = base64_encode(signature, signature_len);
    // string signed_message(reinterpret_cast<char *>
(signature), signature_len);
    free(signature);
    return signed_message;
}

bool RSA_signature_verify(string message, string signatrue)
{
    load_RSA_keys();
    unsigned char hash[SHA256_DIGEST_LENGTH];
    SHA256((unsigned char*)message.c_str(), message.length(), hash);

    size_t decoded_length;
    unsigned char* decoded_signature = base64_decode(signatrue,
&decoded_length);

    if (RSA_verify(NID_sha256, hash, SHA256_DIGEST_LENGTH, decoded_signature,
decoded_length, rsa_public_key) != 1)
    {
        // fprintf(stderr, "RSA signature verification failed\n");
        free(decoded_signature);
    }
}

```



```
        return false;
    }
    free(decoded_signature);
    return true;
}
```

5. 对称加密与哈希

5.1 RC4 (流密码)

记忆： `set_key` 初始化，然后直接异或。

C++

```
string rc4_encrypt(string data, string secret_key)
{
    if (data.empty() || secret_key.empty()) return "";

    RC4_KEY key;
    unsigned char* data_bytes = (unsigned char*)data.c_str();
    unsigned char* key_bytes = (unsigned char*)secret_key.c_str();

    vector<unsigned char> res(data.size());

    RC4_set_key(&key, secret_key.size(), key_bytes);
    RC4(&key, data.size(), data_bytes, res.data());

    return string(res.begin(), res.end());
}

string rc4_decrypt(string data, string secret_key)
{
    return rc4_encrypt(data, secret_key);
}
```

5.2 DES (块密码)

记忆： 准备8字节块 -> `set_key` -> `ecb_encrypt`。

```

void convert_string_to_des_block(const string& str, DES_cblock* des_block)
{
    size_t len = str.size();
    memset(des_block, 0, sizeof(DES_cblock));
    for (size_t i = 0; i < len; i++)
    {
        (*des_block)[i] = str[i];
    }
}

string des_encrypt(string plain, string secret_key)
{
    DES_cblock key, plaintext, ciphertext;
    DES_key_schedule schedule;

    convert_string_to_des_block(secret_key, &key);
    convert_string_to_des_block(plain, &plaintext);

    DES_set_key_checked(&key, &schedule);
    DES_ecb_encrypt(&plaintext, &ciphertext, &schedule, DES_ENCRYPT);

    string result((char*)ciphertext, 8);
    return result;
}

string des_decrypt(string cipher, string secret_key)
{
    DES_cblock key, ciphertext, plaintext;
    DES_key_schedule schedule;

    convert_string_to_des_block(secret_key, &key);
    convert_string_to_des_block(cipher, &ciphertext);

    DES_set_key_checked(&key, &schedule);
    DES_ecb_encrypt(&ciphertext, &plaintext, &schedule, DES_DECRYPT);

    string result((char*)plaintext, 8);
    return result;
}

```


记忆：一行代码搞定。

C++

```
#include <openssl/sha.h>

string calc_sha1(string msg) {
    unsigned char hash[SHA_DIGEST_LENGTH]; // 定义20字节的数组
    // 核心计算函数
    SHA1((unsigned char*)msg.c_str(), msg.size(), hash);

    return string((char*)hash, SHA_DIGEST_LENGTH);
}
```